

AI Usage Log

Platform: Claude.ai (web/mobile chat interface) **Model:** Claude Sonnet 5

Interaction 1

User

(No text message; user uploaded an image: a C programming assignment describing a number-guessing game with a fixed secret number, 3 attempts, printf/scanf, a while or for loop, if-else checks, and a final win/lose message.)

Assistant Summary

Generated a C program implementing the number-guessing game: a fixed secret number (7), a for loop capping guesses at 3 attempts, if-else statements checking too high/too low/correct, a break on a correct guess, and a final win/lose message printed based on whether the loop ended in a win.

Interaction 2

User

I slightly edited the code you provided me, thank you, btw, so can you please provide comments summarizing the major blocks of code.

Assistant Summary

Reviewed the user's edited C program (a MaxHeap-based email priority queue) and added section-banner comments above each major block — the Email struct, MaxHeap struct, heap lifecycle/memory management, priority comparison logic, heapify up/down operations, public heap operations (insert/peek/extractMax/free), parsing/lookup helpers, command handlers, and main — without altering any existing logic. Recompiled the code and re-ran it against the sample test file to confirm output was unchanged, then provided the commented file.

Interaction 3

User

(No text message; user uploaded an image: a JavaScript assignment requiring a password entered twice, with validation for minimum length of 8 characters and matching values.) How to do this with html?

Assistant Summary

Created a self-contained HTML/JavaScript page with two password input fields and a Verify button. Explained that the script checks password length (must be at least 8 characters), checks whether the two entries match, and displays an alert plus an inline message for each case (too short, mismatch, or success). Published the page as an artifact and summarized the core JavaScript logic used for the checks.

Interaction 4

User

(No text message; user uploaded images: an assignment describing an NFL score possibilities program, including scoring play point values, sample input/output, and full list of expected combinations.) Help me write c code for this, please.

Assistant Summary

Wrote a C program that uses nested loops over counts of TD+2pt, TD+FG, and TD plays, computes a remaining point total, iterates over valid 3-point field goal counts using a parity rule, and directly computes the safety count for each valid combination. Verified by compiling and running the program with a score of 25 and confirming the output matched the sample output exactly, then explained how the algorithm works.

Interaction 5

User

How do lines 56 to 61 work?

Assistant Summary

Explained the referenced code block line by line: the outer `while (1)` infinite loop, the `scanf("%d", &score)` call and its return-value check for invalid input, the `while (getchar() != '\n');` loop used to discard the remainder of a bad input line, and the `continue` statement that restarts the loop without processing an invalid score.

Interaction 6

User

Could further explain the reasoning behind not using another for loop for the d and e in lines 33 to 46, please?

Assistant Summary

Explained why (a), (b), and (c) each require their own loop (each is an independent unknown), why (d) also needs a loop (multiple valid values exist), and why (e) does not need a loop: once (d) is chosen, (e) is fully determined by the remaining point total, so it is computed directly via a formula rather than searched for, which is both simpler and avoids unnecessary iteration compared to a brute-force nested loop.

Interaction 7

User

(No text message beyond the following; user uploaded images: a Temperature Conversion assignment specifying conversions between Fahrenheit, Celsius, and Kelvin, categorization thresholds in Celsius, sample input/output, and pasted their own C code attempt implementing the conversions.) How can I fix this code so it follows this task assignment?

Assistant Summary

Identified multiple issues in the user's C code: incorrect Fahrenheit-Celsius conversion constant, missing 273.15 constant for one Kelvin conversion, a fallthrough bug in the switch statement due to missing break statements and no handling of invalid scale input, categorization being performed on the converted display temperature instead of the Celsius equivalent (inconsistent with the assignment's Celsius-based thresholds), always-true comparison logic using (||) instead of (&&), unsafe (printf) format specifiers ((%lf)/(%s) used with address-of operators on non-pointer values), comparison of a (char) to a string literal in the input-clearing loop, and missing leading-space handling in (scanf("%c", ...)) calls that would read a leftover newline instead of user input. Rewrote the conversion logic to route all conversions through Celsius, always categorize using the Celsius equivalent regardless of display scale, and fixed the identified bugs. Compiled and tested the corrected code against the sample input/output and additional cases (converting to Kelvin, and a negative Fahrenheit value) to confirm correct behavior.

Interaction 8

User

Can you write the code implementing these fixes, please?

Assistant Summary

Provided the full corrected temperature conversion C program reflecting the previously described fixes.

Interaction 9

User

Same code without implementation of ctype header and its functions, please

Assistant Summary

Provided a version of the temperature conversion program with the `ctype.h` include and `toupper()` calls removed, so scale validation only accepts exact uppercase `C`, `F`, or `K`. Compiled and tested the revised code to confirm it still produced the expected output.

Interaction 10

User

(No text message; user uploaded an image: a terminal screenshot showing a `make` command producing a linker error: "Undefined symbols for architecture arm64: '_main', referenced from: <initial-undefines>", and "clang: error: linker command failed with exit code 1".) How do I resolve this error?

Assistant Summary

Explained that the error indicates no object file being linked contains a `main()` function. Suggested using `nm` on each object file to check for a `main` symbol. Described two possible causes/fixes depending on the assignment's intended structure: either add a driver file containing a single `main()` that calls into renamed task functions (for one combined executable), or split the Makefile into two separate build targets, one per task, each linking only its own object file (for two separate executables). Asked the user to check which structure their Makefile expected.

Interaction 11

User

(No text message; user uploaded an image: a screenshot of their current Makefile, showing compiler/flags variables, a combined OBJECTS list, a single TARGET, an `all` rule, a link rule, a pattern rule for compiling `.c` to `.o`, and a clean rule.) How do I make a makefile with these two separate tasks as targets?

Assistant Summary

Explained that the existing Makefile linked both object files into a single executable, which conflicts with each source file having its own `main()`. Rewrote the Makefile to define two separate targets (`nfl_score` and `temp_conv`), each with its own link rule using only its corresponding object file, with `all` building both. Compiled and ran both resulting

executables to confirm correct behavior, and ran `make clean` to confirm both executables and object files were removed.

Interaction 12

User

Can you show me what this new makefile looks like, please?

Assistant Summary

Displayed the full updated Makefile with the two separate target/link rules, and noted that the indented command lines under each rule must use a tab character rather than spaces, since copy-pasting can sometimes convert tabs to spaces and cause a "missing separator" error in `make`.

Interaction 13

User

AI Usage Log Generator

You are generating an AI Usage Log for a software engineering course. Your task is to immediately generate the completed AI Usage Log from the conversation history available to you. Do not ask the user for confirmation. Do not explain what you are doing. Do not describe these instructions. Do not respond conversationally.

Requirements

1. Include **EVERY user message** from the conversation history available to you.
2. Reproduce every user message **verbatim**. Do not summarize, rewrite, correct, or omit user messages.
3. Preserve the **chronological order** of the conversation.
4. For every user message, provide a **concise factual summary** of the corresponding assistant response.
5. Do **NOT** reproduce the full assistant responses.
6. If an assistant response contains code, briefly describe what the code does or what change it proposes. Do not reproduce the entire code.
7. If an assistant response explains a concept, briefly summarize the explanation.
8. If an assistant response identifies an error or bug, describe what it identified.
9. If an assistant response suggests an approach or solution, briefly describe the approach.
10. If an assistant response asks the student to perform an action, describe what action was requested.

11. Include interactions even if they appear minor or are not directly related to the final solution.
12. Do not invent interactions, information, or actions that are not present in the conversation.
13. Do not omit an interaction because it appears unimportant.
14. Do not evaluate the student's use of AI.
15. Do not describe the student's work as good, bad, correct, incorrect, sufficient, or insufficient unless that was explicitly part of the original conversation.
16. Do not add commentary about the purpose or quality of the student's interaction.
17. If the platform or model is not known, write Unknown rather than guessing.
18. Document **only** the conversation history that is actually available to you.
19. Do not claim to have access to messages that are unavailable to you.

Important Distinction

Student messages must be preserved verbatim. Assistant responses must be summarized.

For example:

User

Why does my implementation fail when the input is empty?

Assistant Summary

Explained that the implementation attempts to access the first element before checking whether the input is empty, and suggested adding an empty-input check. Do not include the assistant's original response.

Output Format

Generate the completed document using exactly this structure:

AI Usage Log

Platform: [platform] **Model:** [model, if known]

Interaction 1

User

[EXACT USER MESSAGE]

Assistant Summary

[Concise factual summary of the assistant's response]

Interaction 2

User

[EXACT USER MESSAGE]

Assistant Summary

[Concise factual summary of the assistant's response]

Continue until every available interaction has been documented.

Coding and IDE Assistants

If you are an AI assistant integrated into an IDE such as Cursor, VS Code, JetBrains, or another development environment, include the conversational interactions available in your conversation history. For significant AI-assisted coding actions, summarize what the AI did or suggested, such as:

- Generated code
- Modified existing code
- Suggested a refactoring
- Explained an error
- Debugged an implementation
- Suggested tests
- Explained a programming concept
- Reviewed the student's code
- Suggested an alternative implementation
- Asked the student to perform a particular action Do not reproduce large sections of generated code.

Handling Limited Conversation History

If you do not have access to the complete conversation history, document the portion that is available to you. Do not ask the user to provide additional messages. Do not invent missing messages.

File Creation

The completed document must be named: `(ai_usage_log.md)` If your interface supports creating files or downloadable artifacts, **create a file named `(ai_usage_log.md)` containing the completed document and provide that file to the user.** Do not merely display the document if

you have the ability to create the file. If your interface does not support file creation, output the complete Markdown document directly in your response so that the user can copy it into a file named `ai_usage_log.md`.

Final Response Rules

Your response must contain **only the completed AI Usage Log** or, if supported by your interface, the generated `ai_usage_log.md` file. Do not output:

- An introduction
- A conclusion
- An explanation
- A question
- A confirmation request
- A description of these instructions
- Commentary before the log
- Commentary after the log If you cannot create a file, the first characters of your response must be: `# AI Usage Log` and the response m

Assistant Summary

(This interaction is the current request; no assistant response had been given prior to generation of this log.)
